《计算机程序的构造和解释》期中考试

（2025-2026 秋季学期，授课教师：冯新宇、李樾，考试方式：半开卷，只允许携带一张 A4 纸资料）

学号：_____ 姓名：_____ 院系：_____ 得分：____（满分 100）

题号	1	2	3	4	5	6	7	8	总分
得分									

注意事项（请仔细阅读）：

1. 试题的难度不是线性递增的，**注意时间分配**。在这项测验中，可能有些题目较难，因此你不要在一道题上思考时间太久，遇到不会答的题目可先跳过，如果有时间再去思考。否则，你可能没有时间完成后面的题目。
2. 代码填空题的**正确答案不止一种**。代码填空题答案正确即可得分，错误也不会倒扣分。每空**最多只能填一个** Python 语句或表达式，可以直接使用的 Python 内置函数（Python Built-in Functions）包括：`all`，`any`，`bool`，`dict`，`enumerate`，`filter`，`float`，`int`，`iter`，`len`，`list`，`map`，`max`，`min`，`pow`，`print`，`range`，`reversed`，`round`，`sorted`，`sum`，`zip`。

得分	
评分人	

1 WWPD: What Would Python Display? (15 pts)

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. Each expression has at least one line of output.

- If an error occurs, write **`Error`**, but include all output displayed before the error.
- To display a function value, write **`Function`**. To display an iterator or generator value, write **`Iterator`**.
- If an expression would take forever to evaluate, write **`Forever`**.

```
def counted_fib(n):
    count = 0
    def fib(n):
        nonlocal count
        count += 1
        if n <= 1:
            return n
        else:
            return fib(n - 1) + fib(n - 2)
    result = fib(n)
    return (count, result)

def mystery():
    yield 1
    yield 1
    iter1 = mystery()
    iter2 = mystery()
    next(iter2)
    yield from map(lambda pair: pair[0] + pair[1], zip(iter1, iter2))
```

Expression	Interactive Output
`sum([i**3 for i in range(1, 10)])`	2025
`print(7, 8, 9) * 6`	7 8 9 Error
`print` `print("hello") and print("world")`	

`lst1 = [1, 2, 3]`
`lst2 = lst1 * 2`
`lst3 = [lst1] * 2`
`lst1[1] = 10`
 	
`lst1`
`lst2`
`lst3`	

`lst = list(range(1, 5))`
 	
`lst`
`lst[1:-1]`	

`counted_fib(0)`
`counted_fib(1)`
`counted_fib(2)`
`counted_fib(3)`	

`series1 = mystery()`
`series2 = filter(lambda n: n % 2 != 0, series1)`
 	
`[next(series1) for _ in range(2)]`
`[next(series1) for _ in range(2)]`
`[next(series2) for _ in range(2)]`
`next(series1)`	

得分	
评分人	

2 Environment Diagrams (10 pts)

Fill in the environment diagram that results from executing the code on the right until the entire program is finished, an error occurs, or all frames are filled.

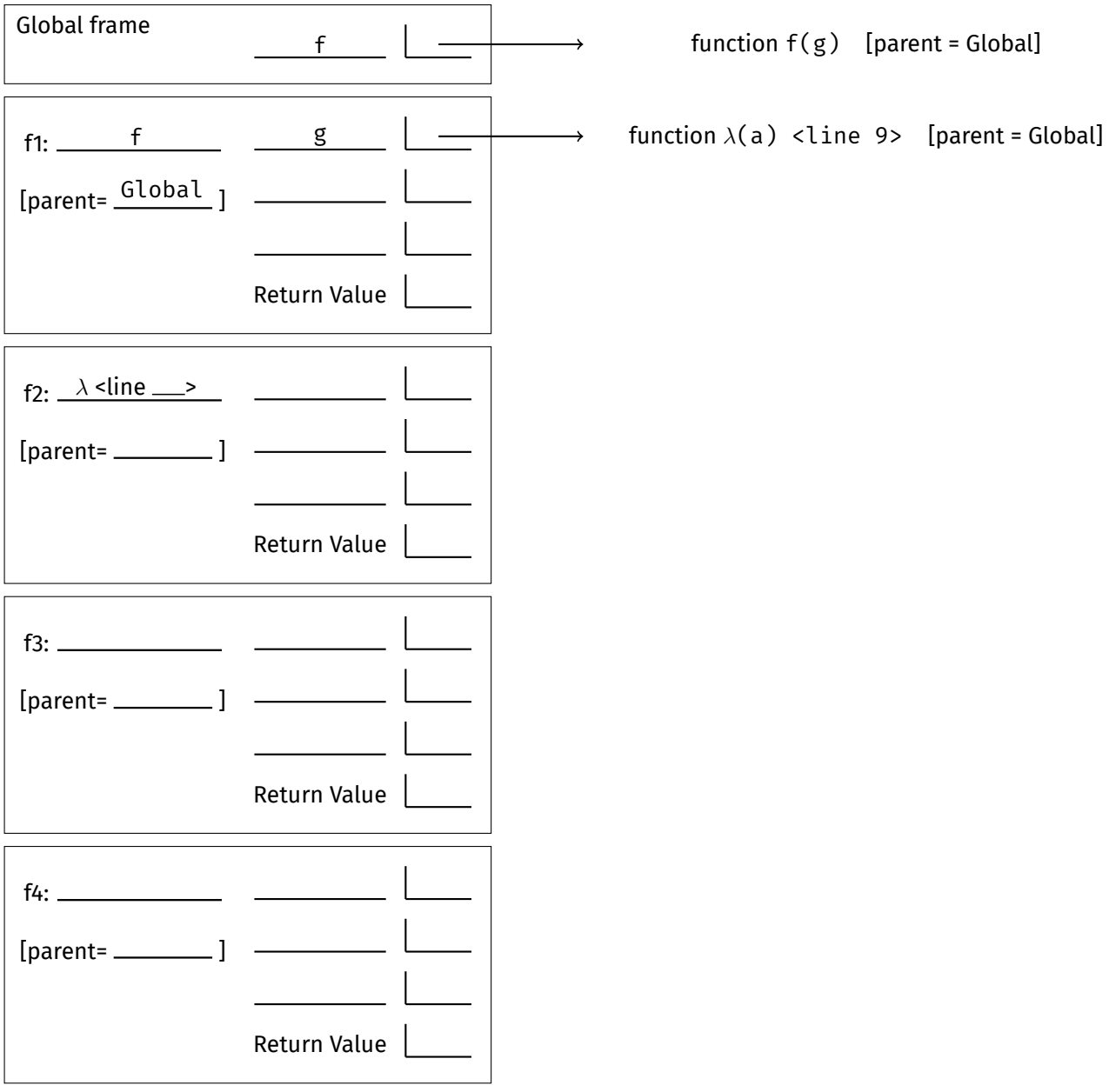
A complete answer will:

- Add missing names and parents to all local frames.
- Add missing values created or referenced.
- Show the return value for each local frame.

Hint:

- **You may not need to use all of the spaces or frames.**
- **If a variable is assigned more than once, draw its last value.**

```
1  def f(g):
2      a = 1
3      def h(b):
4          nonlocal a
5          a = 2
6          return g(a) * b
7      return lambda c: h(c + a)
8
9  f(lambda a: a + 3)(4)
```



得分	
评分人	

3 Lists (9 pts)

In Python, we can use lists of lists to represent matrices. For example, a 2×2 matrix $\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ can be represented as a list containing 2 lists of integers, `[[1, 1], [0, 1]]`. Here each list corresponds to one row from the original matrix. Please complete the following functions that perform various operations to matrices. For simplicity, we guarantee that all matrices are square matrices (that is, all matrices are $n \times n$ matrices for some positive integer n). For an arbitraty matrix A, we use $A_{i,j}$ to denote the element in the i-th row and j-th column.

The **trace** of a matrix is the sum of all the diagonal values of the matrix. In short, $trace(A) = \sum_{i=1}^{n} A_{i,i}$. (2pts)

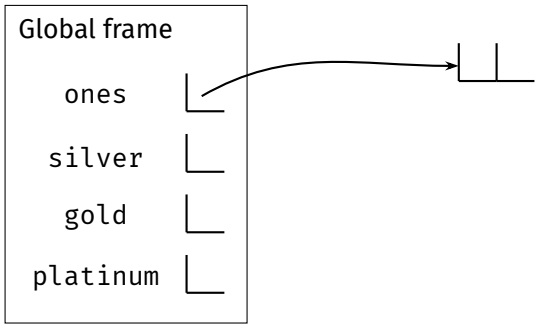
```
def trace(mat):
    """Return the trace of a matrix mat.
    >>> trace([[1, 1], [0, 1]])
    2
    """
    return ____________________________________________________________
```

The **transpose** of a matrix switches the rows and columns of a matrix, flipping it over its main diagnonal. For example, the transpose of a $n \times n$ matrix, A, is defined as $A^{\intercal}{}_{i,j} = A_{j,i}$, for all $1 \leq i, j \leq n$. (2pts)

```
def transpose(mat):
    """Return the transpose of the matrix mat using list comprehension.
    >>> transpose([[1, 2], [3, 4]])
    [[1, 3], [2, 4]]
    """
    return ____________________________________________________________
```

Draw the final environment diagrams for the following code. If an element in the list is assigned multiple times, draw its last value. Think twice before you write! (5pts)

```
ones = [1, 1]
silver = [ones, ones]
gold = silver[:0] + silver[1:]
ones[0] = 6
platinum = [[silver[1][0], silver[0]], [0, 1]]
```



得分	
评分人	

4 One Function for All ! (12 pts)

4.1 Lots of Functions (4 pts)

You have seen many useful (higher-order) functions for lists in this course, such as:

- `map(f, lst)` — returns a list obtained by applying `f` to each element in `lst`;
- `filter(p, lst)` — returns a list containing only the elements of `lst` that satisfy the predicate `p`;
- `reversed(lst)` — returns a list containing the elements of `lst` in reverse order.

Let's meet two new friends from the same family: `foldl` and `foldr`. Fill in the blanks below to complete their recursive implementations based on the given docstrings.

```
def foldl(f, z, lst):
    """ The left fold function foldl(f, z, lst) on an n-element list
    computes f(...f(f(z, lst[0]), lst[1])..., lst[n-1]). For example:
    >>> foldl(add, 0, [1, 2, 3]) # (((0 + 1) + 2) + 3)
    6
    >>> foldl(sub, 0, [3, 2, 1]) # (((0 - 3) - 2) - 1)
    -6
    """
    return foldl(f, ____________________, __________) if lst else z
```

```
def foldr(f, z, lst):
    """ The right fold function foldr(f, z, lst) on an n-element list
    computes f(lst[0], f(lst[1], ... f(lst[n-1], z) ...)). For example:
    >>> foldr(add, 0, [1, 2, 3]) # (1 + (2 + (3 + 0)))
    6
    >>> foldr(sub, 0, [3, 2, 1]) # (3 - (2 - (1 - 0)))
    2
    """
    return f(__________, ____________________) if lst else z
```

4.2 Maybe Not So Many Functions ... (6 pts)

It turns out that `foldl` and `foldr` are powerful enough to express all of our familiar functions — `map`, `filter`, and `reversed` — using only a single function call! Let's see how.

As a warm-up, reimplement `map` and `filter` using `foldl` and `foldr`, respectively.

```
def map(f, lst):
    """ Reimplement map using a single call to foldl.
    >>> map(lambda x: x * 2, [1, 2, 3, 4])
    [2, 4, 6, 8]
    """
    return foldl(_______________________________, [], lst)
```

```
def filter(p, lst):
    """ Reimplement filter using a single call to foldr.
    >>> filter(lambda x: x % 2 == 0, [1, 2, 3, 4])
    [2, 4]
    """
    return foldr(_____________________________, [], lst)
```

Now that you've mastered these, reimplement `reversed` using either `foldl` or `foldr`.

```
def reversed(lst):
    """ Reimplement reversed using a single call to foldl.
    >>> reversed([1, 2, 3, 4])
    [4, 3, 2, 1]
    """
    return foldl(_____________________________, [], lst)
```

```
def reversed(lst):
    """ Reimplement reversed using a single call to foldr. """
    return foldr(_____________________________, [], lst)
```

4.3 Just One Function ! (2 pts)

We've now seen that many useful functions can be built from just `foldl` and `foldr`. But do we really need both? Surprisingly, we can define one using the other!

Fill in the blanks below to express each fold in terms of the other.

```
def foldl(f, z, lst):
    """ Reimplement foldl using a single call to foldr. """
    def step(x, g):
        return ______________________________
    return foldr(step, lambda x: x, lst)(z)
```

```
def foldr(f, z, lst):
    """ Reimplement foldr using a single call to foldl. """
    def step(g, x):
        return ______________________________
    return foldl(step, lambda x: x, lst)(z)
```

得分	
评分人	

5 Don't Break The Fourth Wall (19 pts)

5.1 Introduction (4 pts)

In this problem, we define an abstract data type (ADT) called `lst` to simulate the behavior of Python lists. Read the provided code carefully to understand how the `lst` ADT is implemented using higher-order functions. The insight of the implementation is that a list is an ordered collection of items that can be viewed as a function from indices to items. For example, the list `[1, 4, 5]` can be viewed as a function f with $f(0) = 1$, $f(1) = 4$, and $f(2) = 5$.

We define the four functions below (`lst_empty`, `lst_add`, `lst_len`, and `lst_get`) as the interface of the `lst` ADT. That is, though the evil TAs may later replace the implementation of these functions using other things rather than lambdas, the behavior of these functions will remain as specified in their docstrings.

First, check your understanding of the code by implementing the `lst_get` function in the code snippet below.

```
def lst_empty():
    """Return an empty lst."""
    return lambda i : None

def lst_add(l, item):
    """Return a new lst with item added to the front of the lst l."""
    return lambda i : item if i == 0 else l(i - 1)

def lst_get(l, index):
    """Return the item at position index in the lst l."""
    return None if index < 0 else ________

def lst_len(l):
    """Return the length of the lst l."""
    index = 0
    while lst_get(l, index) is not None:
        index += 1
    return index
```

Then, check your understanding of the code by answering the following WWPD (What Would Python Display?) questions. For each expression, write down the displayed value when the expression is evaluated in a Python interpreter.

Expression	**Interactive Output**
`lst_len(lst_empty())`	`0`
`lst_len(lst_add(lst_add(lst_empty(), 1), 2))`	
`lst_len(lst_add(lst_add(lst_empty(), None), 2))`	
`lst_len(lst_add(lst_add(lst_empty(), 1), None))`	

Note that for simplicity, you do not need to consider nested lists in this problem, that is, lists containing other lists as items (e.g., `[1, [2, 3], 4]`).

5.2 Find the Wall Breakers (10 pts)

ADT separates its interface from its implementation. It is essential not to break the abstraction barrier of an ADT, that is, not to rely on the internal implementation details of the ADT rather than its interface.

The following examples implement some common list operations using the `lst` ADT. Your task is to identify the good implementations that (1) correctly implement the specified functionality and (2) do not break the abstraction barrier of the `lst` ADT. For each implementation, write an "T" if the implementation is good; otherwise, write an "F".

Implementation	Is Good?

`def lst_contains(l, item):`
`  """Return True if item is in the lst l, False otherwise."""`
`  for i in range(lst_len(l)):`
`  if l(i) == item:`
`  return True`
`  return False`	

`def lst_reverse(l):`
`  """Return a new lst that is the reverse of the lst l."""`
`  result = lst_empty()`
`  for i in range(lst_len(l)):`
`  result = lst_add(result, lst_get(l, i))`
`  return result`	

`def lst_append(l, item):`
`  """Return a new lst with item added to the end of l."""`
`  return lambda i : item if i == lst_len(l) else lst_get(l, i)`	

`def lst_to_list(l):`
`  """`
`  Convert all lst abstraction to Python lists, and return a Python list`
`  containing all items in the lst l with the order of items preserved.`
`  """`
`  return [lst_get(l, i) for i in range(lst_len(l))]`	

`def lst_filter(l, f):`
`  """`
`  Return a new lst that only keeps the items in the lst l that`
`  make the function f(item) returns True, and preserve their order in l.`
`  """`
`  result = lst_empty()`
`  for i in range(lst_len(l)):`
`  if f(lst_get(l, i)):`
`  result = lst_add(result, lst_get(l, i))`
`  return result`	

5.3 I Can Do Better than the Noobs (5 pts)

In the previous problem, you have identified many bad implementations written by the TAs. Rather than reviewing their bad code, you may already come up with the idea that "I can do better than those noobs!".

Now it's your turn to solve some problems using the `lst` ADT. Note that the code should (1) correctly implement the specified functionality and (2) not break the abstraction barrier of the `lst` ADT. You need to fill in the blanks in the following code snippets such that the code meets the two requirements above.

First, implement another version of the `lst_append` function in the previous problem:

```
def lst_append(l, item):
    """Return a new lst with item added to the end of l."""
    result = ____________________________
    for i in _______________________________:
        result = lst_add(result, lst_get(l, i))
    return result
```

Then, use the `lst_append` function you just implemented to solve a more complex problem:

```
def lst_interleave(l1, l2):
    """
    Given two lsts l1 and l2, return a new lst that interleaves items from l1 and l2.
    If one lst is longer than the other, append the remaining items at the end.
    For example,
        lst_interleave(
            lst_add(lst_add(lst_empty(), 1), 1),
            lst_add(lst_add(lst_add(lst_empty(), 2), 2), 2))
    would return a lst equivalent to
        lst_add(lst_add(lst_add(lst_add(lst_add(lst_empty(), 2), 2), 1), 2), 1)
    """
    result = lst_empty()
    i, j = 0, 0
    while True:
        if ____________________________________________________:
            return result
        if ____________________________________________________:
            result = lst_append(result, lst_get(l1, i))
            i += 1
        if ____________________________________________________:
            result = lst_append(result, lst_get(l2, j))
            j += 1
```

得分	
评分人	

6 Check-GPT: Generate Powerful Tests (18 pts)

You may need the following tree APIs in this problem:

- `tree(label, branches=[])`
- `label(tree)`
- `branches(tree)`
- `copy_tree(tree)`
- The interactive output of a tree should be the simplest expression that, when evaluated, reconstructs the same tree.

6.1 Why We Need Check-GPT (6pts)

All programmers make mistakes. We write buggy code throughout our coding life. Often, we spend hours afterwards on debugging these programs, just to find out where the bug is hiding. It is not uncommon that we have to read through our programs repeatedly, hoping to understand their complex logic. This helps us find the *corner cases* of our programs, which are unfortunately usually hidden in the OJ by our evil TAs.

For example, take a look at one of our programming assignments, `number_of_k` (doctests are omitted here as we assume that you are all familiar with this problem). You are given the following implementation of this very homework assignment. Assume the code on the left is executed before entering the interactive Python environment. Please complete the WWPD problems on the right. (3pts)

```
1  def number_of_k(n, k):
2      """Return the number of occurrences of k
3      in each digit of a non-negative integer n
4      """
5      count = 0
6      while n > 0:
7          if n % 10 == k:
8              count += 1
9          n = n // 10
10     return count
```

Expression	Interactive Output
`pow(2, 11) - 23`	`2025`
`number_of_k(22000130, 2)`	
`number_of_k(22000130, 0)`	
`number_of_k(0, 0)`	

Another example is taken from hw04, `add_trees`, where you are asked to write a function that returns a new tree, where the labels of each node are the sum of the labels of the corresponding nodes in the two input trees. Consider the following code posted by one of your classmates in our QQ chat group (again, doctests are omitted). Assume the answer is executed before entering the interactive Python. Please answer the WWPD problems on the right. (3pts)

```
1  def add_trees(t1, t2):
2      len1 = len(branches(t1))
3      len2 = len(branches(t2))
4      br = []
5      for i in range(max(len1, len2)):
6          if i < len1 and i < len2:
7              b1 = branches(t1)[i]
8              b2 = branches(t2)[i]
9          elif len1 <= i < len2:
10             b1, b2 = tree(0), branches(t2)[i]
11         else:
12             b1, b2 = branches(t1)[i], tree(0)
13         br.append(add_trees(b1, b2))
14     return tree(label(t1) + label(t2), br)
```

Expression	Interactive Output
`2020 + 5`	`2025`
`'2020' + '5'`	

`t1 = tree(2020)`
`t2 = tree(5)`
`add_trees(t1, t2)`	

`t1 = tree('Hello!',`
`  [tree('I love~')])`
`t2 = tree('SICP!',`
`  [tree('trees'),`
`  tree('yay')])`
`add_trees(t1, t2)`	

Apparently, the programs above are incorrect on some inputs. However, it is often annoying to manually find out the missing corner case. All programmers are lazy, and we are no exception.

Well, this problem can be solved if we use some clever techniques that we have learnt so far in this course. Since we are programmers (aren't we all?), we will be tackling this dilemma using programs. Instead of understanding what our buggy programs exactly do, we shall turn to other ways of finding the bugs inside.

6.2 Generating Inputs (8pts)

Recall how TAs test your programs — often a number of inputs are generated, either manually or automatically by programs, and then these inputs are passed as arguments to your function written in your homework or lab assignment. The outputs of your program is then checked to see if they are correct, under some correctness requirements.

Thus, our first step will be generating some input values for our programs, `number_of_k` and `add_trees`. To do so, we will define two functions, `generate_n_k` and `generate_trees`, to generate inputs for these two functions, respectively. For simplicity, the functions we define in this problem will return a list of inputs. For `generate_n_k`, the list it returns will contain `(n, k)` pairs, where `n` and `k` will be valid inputs to the function `number_of_k` that we want to test; for `generate_trees`, it will return a list of trees, and we will use these trees to test `add_trees`.

6.2.1 Generating pairs (3pts)

For `number_of_k`, it takes in exactly two parameters, `n` and `k`. To make our functions reusable and composable, we will first define a simpler function, `generate_n(start, f, n, valid)`, that generates `n` integers using a function `f` that generates the next element in the list, satisfying some condition function `valid`. The resulting list would be something like `[start, f(start), f(f(start)), ...]`, and for all the integers `i` in the returned list, `valid(i)` returns `True`.

After we are done with `generate_n`, we can build the function we actually wanted, `generate_n_k`. It returns a list of pairs of `(n, k)` such that `n` and `k` are valid inputs to the function that we want to test, `number_of_k`.

```
def generate_n(start, f, n, valid):
    inputs = []
    current = start
    while ________________________________________:
        if valid(current):
            inputs.append(current)
        ________________________________________
    return inputs
```

```
def generate_n_k(max_n):
    """Return a list containing all possible inputs to number_of_k, where n should
    be less than max_n. You answer must be a single call to generate_n.
    """
    inputs = []
    for i in ____________________________________________________:
        for k in range(10):
            inputs.append((i, k))
    return inputs
```

6.2.2 Generating Trees (5pts)

Compared to generating integer pairs, generating trees is a bit harder. To make this problem easier, we only need to generate *binary trees*. A binary tree is a tree whose nodes either have no branches or have exactly two branches. As you might have guessed, binary trees are also recursively defined, just as the parenthesis sequence:

- For all valid labels, `tree(l)` is a binary tree.
- If `left` and `right` are binary trees, then for all valid labels `l`, `tree(l, [left, right])` is a binary tree.

To make sure that our test generation function does not run forever, we use a parameter, `size` (as shown in the code block below), to limit the size of the generated tree. Specifically, `generate_trees(3, [1, 2])` should return a list containing 8 trees; each tree in the returned list should contain exactly 3 nodes (the root, the left branch of the root, and the right branch of the root, since we are talking about binary trees).

Similarly, `generate_trees(5, ['sicp', '2025'])` should return a list containing _____ trees (1pts).

Now read through the following function, and try to complete it to generate binary trees recursively (4pts).

```
def generate_trees(size, labels):
    """Return a list of all possible binary trees with exactly size nodes, where each
    node's label is chosen from the list of labels.
    """
    if size == 0:
        return []
    elif size == 1:
        return ________________________________________
    else:
        result = []
        for ________________________________________:
            for k in range(1, size):
                left_subtrees = generate_trees(k, labels)
                right_subtrees = ________________________________________
                for left in left_subtrees:
                    for right in right_subtrees:
                        result.append(________________________________________)
        return result
```

6.3 Testing with Generated Inputs (4pts)

6.3.1 Comparing `number_of_k` to an Oracle (1pts)

Since we have generated a bunch of inputs for `number_of_k`, the next step is to test our implementation against these inputs. In the field of software testing, there is a concept called *oracle*, which is a correct program that outputs the expected results for all possible inputs (in our case, the answer written by the TAs). Suppose we are given an oracle called `number_of_k_answer`, we can then generate some inputs using `generate_n_k`, and then compare the outputs of our implementation with the outputs of the oracle on these inputs.

Unfortunately, the oracle is often not available for us. This means we will have to write our own oracle if we want to test the correctness of `number_of_k`. Since we don't have to submit the oracle, we can write it in any way we want, as long as it produces the correct outputs for all possible inputs. Thus, we can write another simpler function, `number_of_k_steal_chicken`, using constructs that violate the requirements of the original problem, such as `str()` and `str.count()`, yet guarantees to produce correct outputs for all possible inputs.

```
def number_of_k_steal_chicken(n, k):
    """An oracle for number_of_k, using str() and str.count()
    >>> str(0)
    '0'
    >>> '123203'.count('2')
    2
    """
    return str(n).count(str(k))
```

Finally, we can pass the generated input pairs to both our implementation and the oracle, and compare their outputs.

```
# Suppose number_of_k and number_of_k_steal_chicken are evaluated and defined
max_n = 100
for n, k in generate_n_k(max_n):
    ____________________________________________________________
    if expected != got:
        print("Error on number_of_k({0}, {1})".format(n, k))
        print("Expected: {0}".format(expected))
        print("But Got: {0}".format(got))
```

6.3.2 Fuzzing `add_trees` (3pts)

The testing for `add_trees` is slightly different, due to the lack of an easy oracle. However, we can still do something meaningful using `generate_trees`. For example, we can check whether our program raises some **Error** when we pass in two valid trees as inputs, as correct programs should never crash or fail unexpectedly on valid inputs.

```
trees = generate_trees(4, ['S', 'I', 'C', 'P'])
for t1 in trees:
    for t2 in trees:
        add_trees(t1, t2)
print("No Errors!")
```

This kind of testing is called *fuzzing*, where we typically generate a large amount of valid inputs to the tested program to see if it is robust enough to function correctly, without raising Errors. However, if we take a closer look at the test we have just defined above, it actually does not generate many trees under some special `size`. For example, `generate_trees(4, ['S', 'I', 'C', 'P'])` will return a list containing _____ trees (1pts).

To fix this problem, we have to generate all the trees containing *at most* `size` nodes, instead of *exactly* `size` nodes, to generate more meaningful trees. Please complete the one-liner implementation below using the `generate_trees` that we just defined earlier (2pts).

```
def generate_trees_at_most(size, labels):
    """Return a list of all possible binary trees with at most size nodes, where each
    node's label is chosen from the list of labels. (Hint: Use sum!)
    """
    return ____________________________________________________________
```

7 Infinite Mathematical Sequences (17 pts)

得分	
评分人	

In this problem, you will progressively construct several infinite mathematical sequences using Python generators, and solve some mathematical problems that you have already encountered before entering college. Let's begin with a simple warm-up task.

7.1 Warming Up — Natural Number (3 pts)

In this task, you need to implement a generator function `naturals` that produces the infinite sequence of natural numbers (begin with 0):

```
def naturals():
    """ Return a generator that yields all natural numbers"""
    x = -1
    while ________:
        __________
        __________
```

Now that you have mastered the basic methods of generating infinite sequences, let's try some slightly more challenging cases.

7.2 A Step Further — Perfect Squares (4 pts)

In this task, you need to implement a generator function `squares` that produces the infinite sequence of squares of natural numbers (begin with 0):

Hint: You may use the generator function you implemented in previous task.

```
def squares():
    """ Return a generator that yields all natural numbers"""
    x = _______________
    while ________:
        yield next(x) * next(x)
```

Now you have implemented `squares`, do you think that this function can achieve our goal? If you do, write `Haha, don't try to deceive me!` below. Otherwise, describe which line is incorrect and how to correct the code:

_____

7.3 Not Only One Generator Function (4 pts)

You've implemented generator functions `naturals` and `squares`. Now you need to put on your thinking cap and recall the formula for summing numbers from 1 to n that you learned in high school, then implement a generator function `summation` that produces the infinite sequence of the cumulative sum.

```
def summation():
    """ Return a generator that yields all cumulative sum from 0
    For example:
    >>> sums = summation()
    >>> next(sums)
    0 # 0
    >>> next(sums)
    1 # 0 + 1
    >>> next(sums)
    3 # 0 + 1 + 2
```

```
    >>> next(sums)
    6 # 0 + 1 + 2 + 3
    """
    x, y = _____________, _____________
    while ________:
        yield ___________________
```

7.4 Evil TA's Triple (6 pts)

A Pythagorean triple is a tuple that consists of positive integers (a, b, c) that satisfies:

$$a^2 + b^2 = c^2 \tag{1}$$

But evil TA thinks it's too common and he modifies the triple. A Evil TA's Triple is a tuple (a, b^2, c^2) in which a, b and c are non-negative integers and it satisfies:

$$a + b^2 = c^2 \tag{2}$$

You need to implement generator functions below to finish evil TA's requirements.

```
def natural_limit(n):
    """ Return a generator that yields first n numbers in naturals()"""
    natural = naturals()
    for i in range(n):
        yield next(natural)

def square_limit(n):
    """ Return a generator that yields first n numbers in squares()"""
    square = _____________
    for i in range(n):
        yield ___________________

def evil_triples():
    """ Return a generator that yields all evil triples For example:
    >>> triples = evil_triples()
    >>> next(triples)
    (0, 0, 0)
    >>> next(triples)
    (1, 0, 1)
    >>> next(triples)
    (0, 1, 1)
    """
    for c in naturals():
        c_s = ___________
        for b_s in square_limit(c + 1):
            for a in ____________________________:
                if _________________________:
                    yield _________________________
```

得分	
评分人	

8 Bonus (3 pts)

1. Which of the following is the correct icon (图标) of our course website (https://sicp.pascal-lab.net/2025)? _____ (1pt).



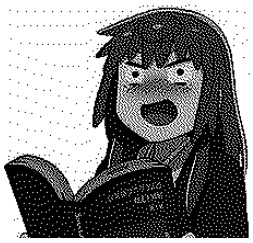
(a)



(b)



(c)



(d)



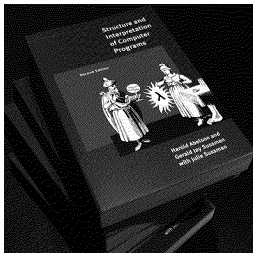
(e)



(f)



(g)



(h)

2. According to the TAs, Hog Contest 2025 is sponsored by _____. (1pts)
3. Among 5 of our TAs, _____ of them have a ponytail (马尾). (1pts)